



Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

LEARNER GUIDE

For

CompTIA Certified Network+ Training (N10-009)

TGS Ref No: TGS-2025054472

Conducted by

Tertiary Infotech Academy Pte Ltd

UEN: 201200696W

Version 1.0

DOCUMENT VERSION CONTROL RECORD

Version Number	Effective Date of Release	Summary of Included Changes	Author
1.0	2 July 2026	Initial release — Network+ (N10-009) Learner Guide covering all 36 labs.	Dr. Alfred Ang

TABLE OF CONTENTS

Right-click and choose “Update Field”, or press F9, to build the table of contents.

Introduction

This Learner Guide accompanies the WSQ course CompTIA Certified Network+ Training (N10-009) (TGS-2025054472), conducted by Tertiary Infotech Academy Pte Ltd. It provides step-by-step instructions for all 36 hands-on labs, organised by the five official CompTIA Network+ N10-009 exam domains. Every lab maps to a published exam objective and is completed on the free Killercoda Ubuntu Playground in your browser.

Use this guide alongside the course slides and the lab files in the labs/ folder of the course repository. Reset the Killercoda playground between labs that change firewall or routing rules so each lab starts from a clean state.

Course Learning Outcomes

- LO1: Explain networking concepts — the OSI model, IPv4/IPv6 addressing and subnetting, ports and protocols, DNS and DHCP, network topologies, architectures and cloud connectivity.
- LO2: Implement networks — static and dynamic routing, 802.1Q VLANs, trunking, link aggregation, NAT/PAT, wireless, and core DHCP and DNS services.
- LO3: Operate and maintain networks — SNMP monitoring, flow analysis, centralized logging and SIEM, time synchronization, configuration backup and performance baselining.
- LO4: Secure networks — firewalls, ACLs, IDS/IPS, TLS certificates, SSH hardening, VPNs, port scanning and access control.
- LO5: Troubleshoot networks — apply the CompTIA seven-step methodology using layer-by-layer diagnostics, connectivity tools and packet capture.

Before You Start — Environment Setup

What you need

- A computer with a modern web browser and internet access.
- The free Killercoda Ubuntu Playground — <https://killercoda.com/playgrounds/scenario/ubuntu> (no local install, no login required).
- For Lab 11 (Wireless): a device with a real Wi-Fi radio and a Wi-Fi analyzer app.
- For Lab 32 (Packet Capture): the browser-based TIS PCAP Analyzer — <https://alfredang.github.io/pcapanalyzer/>.

Launch the Killercoda playground

Open the playground URL in a browser tab. You get a disposable Ubuntu virtual machine with a root shell — nothing to install. Most labs install their own tools with apt at the first step.

```
apt update           # refresh the package index
```

```
ip -brief addr          # confirm your interface (usually eth0) and IP
```

Conventions used in every lab

- Run the shell commands on the Killercoda terminal unless a step says to use a web tool.
- Commands are shown exactly as you type them; output samples follow in the lab files.
- Reset the playground (or run the lab's clean-up step) between labs that change firewall, routing or interface state.
- A subnet/IP calculator is available at <https://alfredang.github.io/ipcalculator/> for the addressing labs.

Domain 1.0 — Networking Concepts (23%)

OSI Model · IPv4/IPv6 Addressing · Ports & Protocols · DNS & DHCP · Topologies & Cloud

Key concepts

- The OSI model's seven layers (Physical → Application) describe how data is encapsulated and de-encapsulated as it crosses a network.
- IPv4 uses 32-bit dotted-decimal addresses with CIDR subnet masks; IPv6 uses 128-bit hexadecimal addresses with SLAAC and link-local addressing.
- Ports and protocols (TCP vs UDP, well-known ports 20–443) identify services; DNS resolves names and DHCP leases addresses via DORA.
- Modern networks span LAN/WAN topologies, three-tier and spine-leaf architectures, and cloud VPCs with public and private subnets.

Lab 1 — OSI Model Packet Inspection

Exam objective: inspect packets and map headers to OSI layers.

Goal: The learner captures live traffic with tcpdump and maps each protocol header to its OSI layer.

What you'll build

A packet capture annotated with the OSI layer of each header (L2 MAC, L3 IP, L4 TCP, L7 HTTP) (Tools & protocols: tcpdump, curl, ip, Berkeley Packet Filter.)

Step-by-step

1. Install the capture tools

```
apt update && apt install -y tcpdump curl
```

2. Identify the active interface

```
ip -brief addr
```

3. Start a background capture on TCP 80

```
tcpdump -i eth0 -nn -v -c 5 'tcp port 80' &
```

4. Generate HTTP traffic

```
curl -s http://example.com > /dev/null
```

5. Read the packet and map it to the OSI model

```
tcpdump -i eth0 -nn -e -A -c 10 'tcp port 80' &
```

6. Stop background captures

```
kill %1 2>/dev/null
```

Test it

Verify you can point at a captured packet and name the OSI layer of each header.

Note: Full commands and expected output are in labs/lab-01-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 2 — IPv4 Subnetting and CIDR

Exam objective: subnet a network into CIDR blocks and verify boundaries.

Goal: The learner splits a /24 network into four /26 subnets, verifies the math in the TIS IP Calculator, and assigns a CIDR address on Linux.

What you'll build

Four /26 subnets with verified network, host range and broadcast, plus an assigned interface IP (Tools & protocols: TIS IP Calculator, ip, CIDR.)

Step-by-step

7. Open the IP Calculator at <https://alfredang.github.io/ipcalculator/>
8. Understand the parent network 192.168.10.0/24 in the calculator
9. Split into four /26 subnets using the calculator
10. Assign an IP from subnet 2 to your interface

```
ip addr add 192.168.10.65/26 dev eth0 label eth0:1
```

11. Verify the address is applied

```
ip -4 addr show eth0
```

12. Clean up the assigned address

```
ip addr del 192.168.10.65/26 dev eth0
```

Test it

Verify a /24 divides into four /26 networks each with 62 usable hosts and a unique broadcast.

Note: Full commands and expected output are in labs/lab-02-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 3 — IPv6 Addressing and Link-Local

Exam objective: plan IPv6 prefixes and distinguish link-local from global addresses.

Goal: The learner plans an IPv6 prefix in the TIS IP Calculator, adds a global address, observes the auto-generated link-local address, and pings across both.

What you'll build

An interface carrying both a link-local fe80::/10 address and a global 2001:db8::10/64 address (Tools & protocols: TIS IP Calculator, ip, ping6, SLAAC.)

Step-by-step

13. Plan the IPv6 prefix 2001:db8::/64 in the calculator at <https://alfredang.github.io/ipcalculator/>
14. Inspect the existing IPv6 configuration

```
ip -6 addr show eth0
```

15. Add a documentation-range global address

```
ip -6 addr add 2001:db8::10/64 dev eth0
```

16. Verify the new address

```
ip -6 addr show eth0
```

17. Ping the all-nodes link-local multicast

```
ping6 -c2 ff02::1%eth0
```

18. Ping the global address from itself

```
ping6 -c2 2001:db8::10
```

19. Clean up the global address

```
ip -6 addr del 2001:db8::10/64 dev eth0
```

Test it

Verify you can distinguish a link-local fe80::/10 address from a global address and ping each.

Note: Full commands and expected output are in labs/lab-03-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 4 — Common Ports and Protocols

Exam objective: map common protocol names to their well-known ports.

Goal: The learner enumerates listening services and scans a live target to connect protocol names to their well-known port numbers.

What you'll build

A verified mapping of protocol names to well-known ports from local listeners and a remote scan (Tools & protocols: nmap, ss, net-tools, getent.)

Step-by-step

20. Install nmap and net-tools

```
apt update && apt install -y nmap net-tools
```

21. List ports listening on your own host

```
ss -tulnp
```

22. Scan a public test target

```
nmap -sT -p 1-1024 scanme.nmap.org
```

23. Map the results to the exam port list

24. Resolve a service name to a port

```
getent services https
```

Test it

Verify you can name the well-known port for each common protocol tested on N10-009.

Note: Full commands and expected output are in labs/lab-04-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 5 — DNS Record Types

Exam objective: query each DNS record type on the exam blueprint.

Goal: The learner uses dig to query every DNS record type in the N10-009 objectives: A, AAAA, MX, TXT, NS, CNAME, PTR, SOA, and SRV.

What you'll build

A set of dig query results covering A, AAAA, MX, TXT, NS, CNAME, PTR, SOA and SRV records (Tools & protocols: dig, dnsutils, DNS.)

Step-by-step

25. Install dnsutils

```
apt update && apt install -y dnsutils
```

26. Query an A record

```
dig +short A google.com
```

27. Query an AAAA record

```
dig +short AAAA google.com
```

28. Query an MX record

```
dig +short MX google.com
```

29. Query a TXT record

```
dig +short TXT google.com
```

30. Query NS records

```
dig +short NS google.com
```

31. Query a CNAME record

```
dig +short CNAME www.github.com
```

32. Reverse lookup a PTR record

```
dig +short -x 8.8.8.8
```

33. Query the SOA record

```
dig +short SOA google.com
```

34. Query an SRV record

```
dig +short SRV _xmpp-server._tcp.gmail.com
```

Test it

Verify you can query each required record type and explain how a CNAME differs from an A record.

Note: Full commands and expected output are in labs/lab-05-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 6 — DHCP Lease Capture (DORA)

Exam objective: capture the four-step DHCP DORA exchange.

Goal: The learner releases an existing DHCP lease and captures the four DORA messages (Discover, Offer, Request, Acknowledge) live with tcpdump.

What you'll build

A live capture showing DHCPDISCOVER, DHCPOFFER, DHCPREQUEST and DHCPACK on UDP 67/68 (Tools & protocols: isc-dhcp-client, dhclient, tcpdump, DHCP.)

Step-by-step

35. Install the tools

```
apt update && apt install -y isc-dhcp-client tcpdump
```

36. Start a capture for DHCP ports

```
tcpdump -i eth0 -nn -v 'port 67 or port 68' &
```

37. Release the current lease

```
dhclient -v -r eth0
```

38. Request a new lease

```
dhclient -v eth0
```

39. Stop the capture

```
kill %1 2>/dev/null
```

Test it

Verify you can identify all four DHCP DORA messages and the UDP ports (67/68) they use.

Note: Full commands and expected output are in labs/lab-06-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 7 — Cloud VPC with Network Namespaces

Exam objective: simulate a cloud VPC using Linux network namespaces.

Goal: The learner builds a VPC with two subnets and a bridge using Linux network namespaces and veth pairs, then tests connectivity between the two virtual machines.

What you'll build

A simulated VPC with two namespaces on a shared /24 subnet connected through a Linux bridge (Tools & protocols: iproute2, ip netns, veth, bridge.)

Step-by-step

40. Create two namespaces representing two VMs

```
ip netns add web
```

41. Create a bridge as the VPC switch

```
ip link add br0 type bridge
```

42. Connect each namespace to the bridge with a veth pair

```
ip link add veth-web type veth peer name br-web
```

43. Assign IPs in the same /24 subnet

```
ip netns exec web ip addr add 10.0.1.10/24 dev veth-web
```

44. Test connectivity between the two VMs

```
ip netns exec web ping -c2 10.0.1.20
```

45. Clean up namespaces and bridge

```
ip netns del web
```

Test it

Verify two hosts on the same subnet can talk through a bridge with no router.

Note: Full commands and expected output are in labs/lab-07-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Domain 2.0 — Network Implementation (20%)

Routing · VLANs & Trunking · Link Aggregation · NAT/PAT · Wireless · DHCP & DNS Services

Key concepts

- Routing moves packets between subnets using static routes and dynamic protocols (RIP, OSPF, EIGRP, BGP) ranked by administrative distance.
- Switching features — 802.1Q VLANs, trunking, link aggregation (LACP), STP and port security — segment and protect the LAN.
- NAT and PAT translate private addresses to public; a DHCP server and authoritative DNS (BIND9) provide core network services.
- Wireless networks use the 2.4/5/6 GHz bands, non-overlapping channels, SSID/BSS/ESS and 802.1X for secure access.

Lab 8 — Static Routing Between Subnets

Exam objective: configure static routes between subnets.

Goal: The learner builds a three-node subnet A - router - subnet B topology and adds the static routes that let the two subnets communicate.

What you'll build

A routed topology with a forwarding router and static routes on each end host. (Tools & protocols: iproute2, network namespaces, veth pairs, sysctl ip_forward.)

Step-by-step

46. Create three network namespaces

```
ip netns add a
```

47. Wire A-R1 and B-R1 with veth pairs

```
ip link add a-r type veth peer name r-a
```

48. Assign IPs in two different subnets

```
ip netns exec a ip addr add 10.1.1.2/24 dev a-r
```

49. Enable IP forwarding on the router

```
ip netns exec r1 sysctl -w net.ipv4.ip_forward=1
```

50. Add static routes on the end hosts

```
ip netns exec a ip route add 10.2.2.0/24 via 10.1.1.1
```

51. Verify connectivity across subnets

```
ip netns exec a ping -c2 10.2.2.2
```

52. Clean up the namespaces

```
for n in a r1 b; do ip netns del $n; done
```

Test it

A ping from subnet A to 10.2.2.2 in subnet B succeeds, proving the static routes forward traffic through the router.

Note: Full commands and expected output are in labs/lab-08-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 9 — 802.1Q VLAN Tagging

Exam objective: create 802.1Q VLAN sub-interfaces.

Goal: The learner loads the 802.1Q kernel module and creates two tagged VLAN sub-interfaces on Linux to separate broadcast domains.

What you'll build

Two VLAN sub-interfaces (eth0.10 and eth0.20) each on its own subnet. (Tools & protocols: iproute2, 8021q kernel module, tcpdump.)

Step-by-step

53. Load the 802.1Q kernel module

```
modprobe 8021q
```

54. Create two VLAN sub-interfaces on eth0

```
ip link add link eth0 name eth0.10 type vlan id 10
```

55. Assign each VLAN its own subnet

```
ip addr add 192.168.10.1/24 dev eth0.10
```

56. Verify the VLAN IDs on the sub-interfaces

```
ip -d link show eth0.10
```

57. Capture a tagged frame with tcpdump

```
tcpdump -i eth0 -nn -e vlan &
```

58. Clean up the VLAN sub-interfaces

```
ip link del eth0.10
```

Test it

tcpdump prints the 802.1Q VLAN tag on frames sent through eth0.10, confirming the frames are tagged with VLAN 10.

Note: Full commands and expected output are in labs/lab-09-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 10 — Link Aggregation (LACP Bond)

Exam objective: bundle NICs into an LACP bond.

Goal: The learner installs ifenslave, loads the bonding driver in 802.3ad mode and creates an LACP bond interface with an enslaved NIC.

What you'll build

A bond0 LACP (802.3ad) interface with an enslaved eth0 slave. (Tools & protocols: ifenslave, bonding kernel module, LACP (IEEE 802.3ad).)

Step-by-step

59. Install ifenslave and load the bonding driver

```
apt update && apt install -y ifenslave
```

60. Create the bond interface in 802.3ad mode

```
ip link add bond0 type bond mode 802.3ad
```

61. Add slave interfaces to the bond

```
ip link set eth0 master bond0
```

62. Inspect the bond status

```
cat /proc/net/bonding/bond0
```

63. Clean up the bond and slave

```
ip link del bond0
```

Test it

Reading /proc/net/bonding/bond0 shows the LACP partner MAC, aggregator ID and per-slave link state, confirming the bond is active.

Note: Full commands and expected output are in labs/lab-10-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 11 — Wireless Concepts and Survey

Exam objective: survey wireless bands, channels and security.

Goal: The learner surveys wireless bands and channels with iw and compares 2.4/5/6 GHz behaviour and WPA security modes.

What you'll build

A basic Wi-Fi site survey plus a channel-to-band and security-mode reference. (Tools & protocols: iw, wavemon, WiFi Analyzer, Wireless Diagnostics.)

Step-by-step

64. List supported PHYs on a wireless device

```
iw list | less
```

65. Scan nearby access points

```
iw dev wlan0 scan | grep -E "SSID|freq|signal|RSN|WPA"
```

66. Map non-overlapping channels to bands

67. Compare WEP, WPA2 and WPA3 security modes

68. Quiz yourself on the wireless concepts

Test it

The learner records SSID, channel, signal and security mode for nearby APs and correctly maps channels to the 2.4/5/6 GHz bands.

Note: Full commands and expected output are in labs/lab-11-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 12 — NAT and PAT with iptables

Exam objective: translate private IPs with NAT and PAT.

Goal: The learner configures an iptables MASQUERADE rule so hosts on a private subnet share the host public IP via port address translation.

What you'll build

A MASQUERADE (PAT) rule on the host translating 10.1.1.0/24 out eth0. (Tools & protocols: iptables, network namespaces, MASQUERADE, conntrack.)

Step-by-step

69. Re-create the private namespace and host link

```
ip netns add a
```

70. Enable forwarding on the host

```
sysctl -w net.ipv4.ip_forward=1
```

71. Add the MASQUERADE rule for PAT

```
iptables -t nat -A POSTROUTING -s 10.1.1.0/24 -o eth0 -j MASQUERADE
```

72. Test outbound access from the namespace

```
ip netns exec a ping -c2 8.8.8.8
```

73. Inspect the conntrack translation table

```
conntrack -L | head
```

74. Clean up rules, namespace and link

```
iptables -t nat -F POSTROUTING
```

Test it

curl ifconfig.me from inside the namespace returns the host public IP rather than 10.1.1.2, proving the source address was translated.

Note: Full commands and expected output are in labs/lab-12-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 13 — DHCP Server (isc-dhcp-server)

Exam objective: configure a DHCP scope and lease server.

Goal: The learner installs isc-dhcp-server and configures a DHCP scope on a dummy interface to hand out leases on a private subnet.

What you'll build

A running ISC DHCP server with a 10.50.0.0/24 scope on a dummy interface. (Tools & protocols: isc-dhcp-server, dummy interface, systemctl.)

Step-by-step

75. Install the ISC DHCP server

```
apt update && apt install -y isc-dhcp-server
```

76. Create a dummy interface for the lab

```
ip link add dum0 type dummy
```

77. Configure the DHCP scope

```
cat >/etc/dhcp/dhcpd.conf <<EOF
```

78. Set the interface the server listens on

```
sed -i 's/^INTERFACESv4=.*/INTERFACESv4="dum0"/' /etc/default/isc-dhcp-server
```

79. Start the DHCP service

```
systemctl restart isc-dhcp-server
```

80. Inspect the leases file

```
cat /var/lib/dhcp/dhcpd.leases
```

81. Clean up the service and interface

```
systemctl stop isc-dhcp-server
```

Test it

systemctl status isc-dhcp-server shows the daemon running and bound to dum0 with the configured scope.

Note: Full commands and expected output are in labs/lab-13-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 14 — Authoritative DNS with BIND9

Exam objective: serve an authoritative DNS zone.

Goal: The learner runs a BIND9 server hosting the authoritative zone lab.local and queries its records with dig.

What you'll build

A BIND9 master server serving the lab.local zone with A, NS and MX records. (Tools & protocols: bind9, dnsutils (dig), named-checkzone.)

Step-by-step

82. Install BIND9 and dnsutils

```
apt update && apt install -y bind9 dnsutils
```

83. Create the lab.local zone file

```
cat >/etc/bind/db.lab.local <<'EOF'
```

84. Register the zone with BIND9

```
cat >>/etc/bind/named.conf.local <<'EOF'
```

85. Validate the zone and reload BIND9

```
named-checkzone lab.local /etc/bind/db.lab.local
```

86. Query your own DNS server with dig

```
dig @127.0.0.1 www.lab.local
```

87. Clean up by stopping BIND9

```
systemctl stop bind9
```

Test it

dig @127.0.0.1 www.lab.local returns the configured A record, confirming BIND9 answers authoritatively for lab.local.

Note: Full commands and expected output are in labs/lab-14-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Domain 3.0 — Network Operations (19%)

SNMP · NetFlow · Syslog & SIEM · NTP · Config Backup · Performance Baselines

Key concepts

- SNMP (traps, MIBs, v3), NetFlow flow data and packet capture give visibility into device health and traffic.
- Centralized logging (syslog / rsyslog) and SIEM correlate events; NTP keeps timestamps consistent across devices.
- Configuration management relies on backups and golden / baseline configs stored in version control (Git).
- Performance baselines (iperf3), documentation (physical / logical diagrams, IPAM) and SLAs keep the network reliable.

Lab 15 — SNMP Monitoring

Exam objective: monitor devices with SNMP.

Goal: The learner installs an SNMP agent, configures a read-only community string, and walks the MIB to read device metrics.

What you'll build

A working snmpd agent with a read-only community, queried via snmpwalk and snmpget (Tools & protocols: SNMP, snmpd, snmpwalk, snmpget, IF-MIB.)

Step-by-step

88. Install agent and client

```
apt update && apt install -y snmpd snmp snmp-mibs-downloader
```

89. Configure a read-only community

```
echo 'rocommunity public 127.0.0.1' >> /etc/snmp/snmpd.conf
```

90. Walk system info

```
snmpwalk -v2c -c public 127.0.0.1 system
```

91. Get a single OID

```
snmpget -v2c -c public 127.0.0.1 sysUpTime.0
```

92. Walk interface counters

```
snmpwalk -v2c -c public 127.0.0.1 IF-MIB::ifDescr
```

Test it

Running snmpwalk against the agent returns system OIDs such as sysDescr, sysName, and sysUpTime.

Note: Full commands and expected output are in labs/lab-15-*.md. Reset the Killercode playground before the next lab if this one changed firewall, routing or interface state.

Lab 16 — NetFlow Analysis with nfdump

Exam objective: export and analyse network flows.

Goal: The learner runs a NetFlow exporter and collector on one VM and analyses the captured flows with nfdump.

What you'll build

A NetFlow exporter (fprobe) feeding a collector (nfcapd) whose flows are reported with nfdump (Tools & protocols: NetFlow, fprobe, nfcapd, nfdump.)

Step-by-step

93. Install the tools

```
apt update && apt install -y nfdump fprobe
```

94. Start the collector

```
nfcapd -p 9995 -l /tmp/flows -D
```

95. Start the exporter

```
fprobe -i eth0 127.0.0.1:9995
```

96. Generate traffic

```
curl -s https://example.com > /dev/null
```

97. Analyse the flows

```
nfdump -R /tmp/flows -s srcip
```

98. Clean up

```
pkill fprobe
```

Test it

nfdump -s builds top-N reports of source IPs and destination ports from the captured flows.

Note: Full commands and expected output are in labs/lab-16-*.md. Reset the Killercode playground before the next lab if this one changed firewall, routing or interface state.

Lab 17 — Centralised Logging (rsyslog)

Exam objective: centralise logs with syslog forwarding.

Goal: The learner configures rsyslog to both send and receive on loopback, proving the log-forwarding pipeline.

What you'll build

An rsyslog TCP receiver on port 514 with a forwarding rule that delivers test events (Tools & protocols: rsyslog, imtcp, logger, syslog TCP/514.)

Step-by-step

99. Enable the TCP receiver

```
sed -i 's/#module(load="imtcp")/module(load="imtcp")/' /etc/rsyslog.conf
```

100. Add a forwarding rule

```
echo '*. * @127.0.0.1:514' > /etc/rsyslog.d/50-forward.conf
```

101. Restart rsyslog

```
systemctl restart rsyslog
```

102. Generate a test event

```
logger -p local0.info "Network+ Lab 17 test event"
```

103. Verify the event landed

```
grep "Lab 17 test" /var/log/syslog
```

104. Inspect severity and facility

Test it

grep finds the injected test event in /var/log/syslog after it traverses the forwarding pipeline.

Note: Full commands and expected output are in labs/lab-17-*.md. Reset the Killercode playground before the next lab if this one changed firewall, routing or interface state.

Lab 18 — Time Sync with Chrony

Exam objective: synchronise clocks with NTP.

Goal: The learner installs chrony, inspects its time sources and tracking state, and forces an immediate sync.

What you'll build

A chrony NTP client synchronising to upstream servers with an added extra source (Tools & protocols: NTP, chrony, chronyc, UDP/123.)

Step-by-step

105. Install chrony

```
apt update && apt install -y chrony
```

106. List your time sources

```
chronyc sources -v
```

107. Show synchronisation state

```
chronyc tracking
```

108. Add an extra NTP server

```
echo "server time.cloudflare.com iburst" >> /etc/chrony/chrony.conf
```

109. Force an immediate sync

```
chronyc -a makestep
```

Test it

chronyc tracking reports the current stratum and clock offset, confirming synchronisation.

Note: Full commands and expected output are in labs/lab-18-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 19 — Configuration Backup with Git

Exam objective: version-control network configuration.

Goal: The learner places /etc under Git version control to gain change history, audit trail, and rollback.

What you'll build

A Git repository in /etc tracking network configs with commit history and a revert (Tools & protocols: git.)

Step-by-step

110. Install git

```
apt update && apt install -y git
```

111. Initialise a repo in /etc

```
git init
```

112. Add critical network configs

```
git add hosts hostname resolv.conf network/ bind/ 2>/dev/null
```

113. Make a change and commit it

```
echo "127.0.1.2 lab-vm" >> /etc/hosts
```

114. Inspect the audit trail

```
git -C /etc log --oneline
```

115. Roll back

```
git -C /etc revert HEAD --no-edit
```

Test it

git log shows the commit history and git revert restores /etc/hosts to its prior state.

Note: Full commands and expected output are in labs/lab-19-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 20 — Performance Baseline with iperf3

Exam objective: baseline throughput, jitter, and loss.

Goal: The learner runs an iperf3 server and client to measure TCP throughput, parallel streams, UDP jitter/loss, and reverse-mode traffic.

What you'll build

An iperf3 server-and-client pair producing TCP, parallel, UDP, and reverse-mode measurements (Tools & protocols: iperf3, TCP, UDP.)

Step-by-step

116. Install iperf3

```
apt update && apt install -y iperf3
```

117. Run a server in the background

```
iperf3 -s -D
```

118. TCP throughput test (10 seconds)

```
iperf3 -c 127.0.0.1 -t 10
```

119. Parallel streams (saturate WAN circuits)

```
iperf3 -c 127.0.0.1 -P 4 -t 10
```

120. UDP test (jitter + loss)

```
iperf3 -c 127.0.0.1 -u -b 100M -t 10
```

121. Reverse mode (server sends to client)

```
iperf3 -c 127.0.0.1 -R -t 10
```

122. Stop the server

```
pkill iperf3
```

Test it

The iperf3 client reports Bitrate for TCP tests and Jitter with Lost/Total Datagrams for the UDP test.

Note: Full commands and expected output are in labs/lab-20-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Domain 4.0 — Network Security (14%)

CIA Triad · Firewalls & ACLs · IDS/IPS · TLS · SSH · VPNs · NAC

Key concepts

- The CIA triad (confidentiality, integrity, availability) and defence-in-depth guide network security design.
- Firewalls, ACLs, screened subnets and IDS/IPS (Suricata) filter and inspect traffic at the perimeter and on the host.
- Encryption secures data in transit — TLS certificates, SSH key authentication and IPsec / WireGuard VPNs (site-to-site, client-to-site).
- Hardening, port scanning (nmap), network access control and key management reduce the attack surface.

Lab 21 — Host Firewall with UFW

Exam objective: configure a host firewall.

Goal: The learner configures a default-deny host firewall with UFW, permitting only required inbound services.

What you'll build

A UFW ruleset that denies all inbound traffic by default and selectively allows SSH and a source-restricted MySQL port. (Tools & protocols: UFW, iptables, SSH, TCP.)

Step-by-step

123. Install UFW

```
apt update && apt install -y ufw
```

124. Set sane default policies

```
ufw default deny incoming
```

125. Allow SSH on port 22

```
ufw allow 22/tcp
```

126. Enable the firewall and view status

```
ufw --force enable
```

127. Add a source-restricted MySQL rule

```
ufw allow from 10.0.0.0/8 to any port 3306 proto tcp
```

128. Delete a rule by number

```
ufw delete 1
```

129. Disable the firewall when done

```
ufw disable
```

Test it

Running `ufw status verbose` confirms the default-deny stance with only the intended allow rules.

Note: Full commands and expected output are in `labs/lab-21-*.md`. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 22 — TLS Certificate with OpenSSL + Nginx

Exam objective: serve HTTPS with a TLS certificate.

Goal: The learner generates a self-signed X.509 certificate, serves it from nginx, and verifies the TLS handshake.

What you'll build

A self-signed RSA-2048 certificate and an nginx HTTPS server confirmed via a client-side TLS handshake. (Tools & protocols: OpenSSL, Nginx, curl, TLS, X.509, HTTPS.)

Step-by-step

130. Install openssl, nginx and curl

```
apt update && apt install -y openssl nginx curl
```

131. Generate a self-signed certificate

```
openssl req -x509 -newkey rsa:2048 -nodes \
    -keyout /etc/ssl/private/lab.key \
    -out /etc/ssl/certs/lab.crt \
    -days 365 \
    -subj "/CN=lab.local"
```

132. Configure nginx for HTTPS

```
cat >/etc/nginx/sites-available/default <<'EOF'

server {

    listen 443 ssl default_server;

    server_name _;
```

```
ssl_certificate      /etc/ssl/certs/lab.crt;

ssl_certificate_key  /etc/ssl/private/lab.key;

root /var/www/html;

index index.html;

}

EOF
```

```
systemctl restart nginx
```

133. Test HTTPS from the client side

```
curl -kv https://localhost 2>&1 | head -30
```

134. Inspect the certificate

```
openssl x509 -in /etc/ssl/certs/lab.crt -noout -text | head -20
```

135. Test the live handshake with s_client

```
echo | openssl s_client -connect localhost:443 -servername lab.local
2>/dev/null | head -20
```

Test it

curl -kv https://localhost shows a successful TLS 1.3 handshake presenting the lab.local certificate.

Note: Full commands and expected output are in labs/lab-22-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 23 — SSH Hardening with Key Auth

Exam objective: harden ssh with key authentication.

Goal: The learner creates an Ed25519 key pair, installs the public key, and disables SSH password authentication.

What you'll build

A key-only SSH configuration using an Ed25519 key pair with password login and root password login disabled. (Tools & protocols: OpenSSH, Ed25519, sshd, public-key authentication.)

Step-by-step

136. Generate an Ed25519 key pair

```
ssh-keygen -t ed25519 -N '' -f ~/.ssh/lab_ed25519
```

137. Install the public key as authorised

```
mkdir -p ~/.ssh && chmod 700 ~/.ssh
```

138. Test key-based login

```
ssh -i ~/.ssh/lab_ed25519 -o StrictHostKeyChecking=no root@localhost whoami
```

139. Disable password authentication

```
sed -i 's/^#*PasswordAuthentication.*/PasswordAuthentication no/'  
/etc/ssh/sshd_config
```

140. Verify password login is blocked

```
ssh -o PreferredAuthentications=password -o PubkeyAuthentication=no  
root@localhost
```

Test it

Attempting password login returns Permission denied (publickey), confirming key-only access.

Note: Full commands and expected output are in labs/lab-23-*.md. Reset the Killercode playground before the next lab if this one changed firewall, routing or interface state.

Lab 24 — Site-to-Site VPN with WireGuard

Exam objective: build a site-to-site vpn tunnel.

Goal: The learner builds a WireGuard tunnel between two Linux network namespaces acting as two VPN endpoints.

What you'll build

A working WireGuard tunnel between site-a and site-b namespaces with peer keys, allowed-ips and connectivity verified by ping. (Tools & protocols: WireGuard, wireguard-tools, Curve25519, ip netns.)

Step-by-step

141. Install WireGuard

```
apt update && apt install -y wireguard wireguard-tools
```

142. Create two namespaces for the two sites

```
ip netns add site-a
```

143. Generate key pairs for each peer

```
wg genkey | tee a.key | wg pubkey > a.pub
```

144. Configure the WireGuard interfaces

```
ip -n site-a link add wg0 type wireguard
```

```
ip -n site-b link add wg0 type wireguard
```

```
ip -n site-a addr add 10.99.0.1/24 dev wg0
```

```
ip -n site-b addr add 10.99.0.2/24 dev wg0
```

```
ip netns exec site-a wg set wg0 \
    private-key /tmp/a.key listen-port 51820 \
    peer "$(cat /tmp/b.pub)" allowed-ips 10.99.0.2/32
```

```
ip netns exec site-b wg set wg0 \
    private-key /tmp/b.key listen-port 51821 \
    peer "$(cat /tmp/a.pub)" allowed-ips 10.99.0.1/32 \
    endpoint 127.0.0.1:51820
```

145. Bring tunnels up and test connectivity

```
ip -n site-a link set wg0 up
```

146. Inspect the tunnel state

```
ip netns exec site-a wg show
```

147. Clean up namespaces and keys

```
ip netns del site-a
```

Test it

`ip netns exec site-b ping -c2 10.99.0.1` succeeds, proving the encrypted tunnel is up.

Note: Full commands and expected output are in labs/lab-24-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 25 — Port Scanning and Hardening

Exam objective: scan ports and reduce attack surface.

Goal: The learner uses nmap to enumerate exposed services on the host and disables the unnecessary ones.

What you'll build

An attack-surface audit produced with nmap and ss, followed by disabling an unneeded service and confirming closure. (Tools & protocols: nmap, ss, systemctl, TCP, SYN scan.)

Step-by-step

148. Install nmap

```
apt update && apt install -y nmap
```

149. Scan all 65535 TCP ports on yourself

```
nmap -sS -p- 127.0.0.1
```

150. Enumerate service versions

```
nmap -sV -p 22,80,443 127.0.0.1
```

151. OS fingerprint the host

```
nmap -O 127.0.0.1
```

152. Cross-check open ports with ss

```
ss -tulnp
```

153. Disable an unneeded service

```
systemctl status apache2 2>/dev/null && systemctl disable --now apache2
```

154. Re-scan to confirm the port is closed

```
nmap -sS -p 80 127.0.0.1
```

Test it

The re-scan with nmap -sS -p 80 shows the disabled service's port is no longer open.

Note: Full commands and expected output are in labs/lab-25-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 26 — Suricata IDS

Exam objective: detect intrusions with an ids.

Goal: The learner installs Suricata, updates its rule set, triggers a test signature, and inspects the resulting alert.

What you'll build

A running Suricata IDS with the Emerging Threats ruleset that logs an alert for benign test traffic. (Tools & protocols: Suricata, jq, Emerging Threats rules, IDS/IPS.)

Step-by-step

155. Install Suricata and jq

```
apt update && apt install -y suricata jq
```

156. Update the Emerging Threats ruleset

```
suricata-update
```

157. Start Suricata on the active interface

```
suricata -i eth0 -D
```

158. Trigger a test signature

```
curl -s http://testmynids.org/uid/index.html >/dev/null
```

159. Inspect the alert log

```
tail /var/log/suricata/fast.log
```

160. Stop Suricata

```
pkill suricata
```

Test it

fast.log and eve.json show a GPL ATTACK_RESPONSE alert triggered by the test traffic.

Note: Full commands and expected output are in labs/lab-26-*.md. Reset the Killercode playground before the next lab if this one changed firewall, routing or interface state.

Lab 27 — ACL with iptables

Exam objective: build a network acl.

Goal: The learner builds an iptables ACL that permits MySQL traffic from one subnet and denies it from everywhere else.

What you'll build

An ordered iptables INPUT chain that accepts port 3306 from 10.0.0.0/24, drops it elsewhere, and is persisted across reboots. (Tools & protocols: iptables, iptables-persistent, netcat, TCP, ACL.)

Step-by-step

161. Inspect current rules

```
iptables -L INPUT -n -v --line-numbers
```

162. Allow MySQL from 10.0.0.0/24 only

```
iptables -I INPUT -p tcp --dport 3306 -s 10.0.0.0/24 -j ACCEPT
```

163. Drop MySQL from everywhere else

```
iptables -A INPUT -p tcp --dport 3306 -j DROP
```

164. Verify rule ordering

```
iptables -L INPUT -n -v --line-numbers
```

165. Test the ACL

```
nc -zv 127.0.0.1 3306          # blocked unless your IP is in 10.0.0.0/24
```

166. Persist the rules

```
apt install -y iptables-persistent
```

167. Clean up the rules

```
iptables -D INPUT -p tcp --dport 3306 -s 10.0.0.0/24 -j ACCEPT
```

Test it

A connection from source 10.0.0.5 behaves differently to other sources, proving the ACL filters by source.

Note: Full commands and expected output are in labs/lab-27-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Domain 5.0 — Network Troubleshooting (24%)

Methodology · Layer 1/2 Diagnostics · Connectivity Tools · DNS/Routing/MTU · Packet Capture

Key concepts

- Follow the seven-step CompTIA methodology: identify the problem → theory → test → plan → implement → verify → document.
- Layer 1/2 issues (cabling, interference, attenuation, port/interface errors) are diagnosed with cable testers and interface counters.
- Connectivity tools — ping, traceroute / mtr, arp, ip / ifconfig, netstat — isolate faults by OSI layer.
- DNS, routing and MTU / PMTUD problems are resolved with dig / nslookup, route inspection and tcpdump / tshark packet capture.

Lab 28 — Layer 1/2 Diagnostics

Exam objective: diagnose layer 1 and 2 faults.

Goal: The learner diagnoses physical and data-link faults using ip, ethtool and kernel interface counters.

What you'll build

A repeatable Layer 1/2 diagnostic workflow reading link state, speed/duplex and error counters (Tools & protocols: ip, ethtool, iproute2.)

Step-by-step

168. Install ethtool

```
apt update && apt install -y ethtool
```

169. Check operational state

```
ip -brief link show
```

170. Inspect link parameters

```
ethtool eth0
```

171. Check error counters

```
ethtool -S eth0 | grep -iE 'err|drop|crc|frame'
```

172. Bounce the interface

```
ip link set eth0 down
```

173. Force speed/duplex (example only, do not run)

```
# ethtool -s eth0 speed 1000 duplex full autoneg off
```

Test it

Verify UP link state with clean CRC/frame counters confirms a healthy Layer 1/2 path.

Note: Full commands and expected output are in labs/lab-28-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 29 — ping, traceroute, mtr

Exam objective: locate loss and latency along a path.

Goal: The learner measures reachability, hop-by-hop path and continuous loss/latency using ping, traceroute and mtr.

What you'll build

A path-diagnosis toolkit measuring RTT, per-hop routers and where loss begins (Tools & protocols: ping, traceroute, mtr, ICMP, TCP.)

Step-by-step

174. Install the tools

```
apt update && apt install -y traceroute mtr-tiny
```

175. Baseline reachability with ping

```
ping -c5 8.8.8.8
```

176. Trace the hop-by-hop path

```
traceroute 8.8.8.8
```

177. Combined live view with mtr

```
mtr -rcw 20 8.8.8.8
```

178. Use TCP traceroute when ICMP is blocked

```
traceroute -T -p 443 example.com
```

Test it

Verify the mtr Loss% column pinpoints where persistent loss starts along the path.

Note: Full commands and expected output are in labs/lab-29-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 30 — DNS Troubleshooting

Exam objective: isolate name resolution problems.

Goal: The learner isolates DNS resolution problems using dig, +trace and resolvectl.

What you'll build

A DNS triage workflow comparing resolvers, walking the delegation chain and decoding error statuses (Tools & protocols: dig, resolvectl, systemd-resolved, dnsutils.)

Step-by-step

179. Install dnsutils

```
apt update && apt install -y dnsutils
```

180. Confirm which resolver the system uses

```
cat /etc/resolv.conf
```

181. Compare two upstream resolvers

```
dig @1.1.1.1 example.com +short
```

182. Trace the recursive resolution chain

```
dig +trace example.com
```

183. Check for slow upstream

```
time dig @1.1.1.1 example.com
```

184. Flush systemd-resolved cache

```
resolvectl flush-caches
```

185. Decode common DNS error statuses

Test it

Verify a resolver returning an IP versus NXDOMAIN identifies the failing DNS server.

Note: Full commands and expected output are in labs/lab-30-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 31 — ARP and Neighbour Table

Exam objective: troubleshoot address-to-mac resolution.

Goal: The learner inspects and manipulates the ARP and IPv6 neighbour tables to resolve stale or duplicate entries.

What you'll build

Neighbour-table diagnostics probing the gateway, detecting duplicate IPs and pinning static entries (Tools & protocols: ip neigh, arping, ndisc6, ARP, ICMPv6 NDP.)

Step-by-step

186. View the current neighbour table

```
ip neigh
```

187. Probe a neighbour with arping

```
arping -I eth0 -c2 $(ip route | awk '/default/ {print $3}')
```

188. Detect duplicate IP

```
arping -D -I eth0 -c2 $(ip -4 -o addr show eth0 | awk '{print $4}' | cut -d/ -f1)
```

189. Add a static ARP entry

```
ip neigh add 10.0.0.99 lladdr de:ad:be:ef:00:01 dev eth0
```

190. Flush the table

```
ip neigh flush all
```

191. Explore IPv6 neighbour discovery

```
ip -6 neigh
```

Test it

Verify arping -D reports a duplicate when another host claims the same IP address.

Note: Full commands and expected output are in labs/lab-31-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 32 — Packet Capture (tcpdump/tshark)

Exam objective: capture and analyse packets.

Goal: The learner captures DNS packets to a pcap and analyses them with tshark and the browser-based TIS PCAP Analyzer.

What you'll build

A saved .pcap of DNS traffic analysed on the CLI and graphically in the TIS PCAP Analyzer (Tools & protocols: tcpdump, tshark, TIS PCAP Analyzer, DNS, BPF.)

Step-by-step

192. Install both tools

```
apt update && apt install -y tcpdump tshark
```

193. Capture 10 DNS packets from this host

```
tcpdump -i eth0 -nn -w /tmp/dns.pcap 'udp port 53 and src host 127.0.0.1'  
-c 10 &
```

194. Read the file with tshark

```
tshark -r /tmp/dns.pcap -Y "dns" -T fields -e frame.time_relative -e  
dns.qry.name -e dns.qry.type
```

195. Filter only response packets

```
tshark -r /tmp/dns.pcap -Y "dns.flags.response==1"
```

196. Generate a friendly summary

```
tcpdump -nn -r /tmp/dns.pcap | head
```

197. Download the .pcap via base64

```
base64 /tmp/dns.pcap > /tmp/dns.pcap.b64
```

198. Analyse the capture in the TIS PCAP Analyzer at
<https://alfredang.github.io/pcapanalyzer/>

199. Answer the quick analysis questions

Test it

Verify the TIS PCAP Analyzer shows all three queries with matching responses and transaction IDs.

Note: Full commands and expected output are in labs/lab-32-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 33 — MTU and Path-MTU Discovery

Exam objective: diagnose mtu and pmtud failures.

Goal: The learner tests interface MTU and simulates Path MTU Discovery using the Don't Fragment ping flag.

What you'll build

MTU tests proving where oversized frames blackhole and how MSS clamping rescues TCP (Tools & protocols: ping, ip link, ICMP, PMTUD.)

Step-by-step

200. See your interface MTU

```
ip -brief link show eth0
```

201. Send a packet at the largest size that fits

```
ping -M do -s 1472 -c2 8.8.8.8
```

202. Force fragmentation needed

```
ping -M do -s 1500 -c2 8.8.8.8
```

203. Lower the interface MTU and retest

```
ip link set eth0 mtu 1400
```

204. Restore the MTU

```
ip link set eth0 mtu 1500
```

205. Understand MTU-mismatch blackholes and MSS clamping

Test it

Verify a DF ping larger than the MTU fails while a fitting payload succeeds.

Note: Full commands and expected output are in labs/lab-33-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 34 — Routing Issues

Exam objective: diagnose and fix routing problems.

Goal: The learner inspects the routing table, queries route selection and adds or removes static routes.

What you'll build

Routing diagnostics using `ip route get`, longest-prefix match and static-route add/remove (Tools & protocols: `ip route`, `traceroute`, `mtr`, `iproute2`.)

Step-by-step

206. Print the full routing table

```
ip route
```

207. Ask the kernel which route a destination uses

```
ip route get 8.8.8.8
```

208. Add a static route

```
ip route add 10.99.0.0/24 via $(ip route | awk '/default/ {print $3}')  
metric 100
```

209. Show route metrics

```
ip route show 10.99.0.0/24
```

210. Detect asymmetric routing

```
traceroute 8.8.8.8
```

211. Remove the test route

```
ip route del 10.99.0.0/24
```

212. Investigate route precedence

Test it

Verify `ip route get` returns the interface, source IP and next-hop chosen for a destination.

Note: Full commands and expected output are in labs/lab-34-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 35 — Network Emulation with tc

Exam objective: emulate delay jitter loss and bandwidth.

Goal: The learner injects delay, jitter, loss and bandwidth limits on an interface with `tc` `netem` and `tb`.

What you'll build

A reproducible WAN emulation adding delay, jitter, loss and a bandwidth cap for app testing (Tools & protocols: `tc`, `netem`, `tb`, `iperf3`, `sch_netem`.)

Step-by-step

213. Baseline ping

```
ping -c5 8.8.8.8
```

214. Add 200 ms of delay

```
tc qdisc add dev eth0 root netem delay 200ms
```

215. Add jitter

```
tc qdisc change dev eth0 root netem delay 200ms 50ms distribution normal
```

216. Add packet loss

```
tc qdisc change dev eth0 root netem delay 200ms loss 5%
```

217. Limit bandwidth with a token bucket filter

```
tc qdisc add dev eth0 root tbf rate 1mbit burst 32kbit latency 400ms
```

218. Remove all shaping

```
tc qdisc del dev eth0 root
```

Test it

Verify ping RTT rises by ~200 ms and iperf3 throughput caps at 1 Mbit/s under shaping.

Note: Full commands and expected output are in labs/lab-35-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Lab 36 — Service Reachability Triage

Exam objective: triage service reachability by layer.

Goal: The learner isolates whether TCP, TLS, HTTP or the app is failing using curl, nc and openssl.

What you'll build

A layered site-down triage testing TCP, TLS, HTTP and per-phase request timing (Tools & protocols: curl, nc, openssl, netcat-openbsd, TCP, TLS, HTTP.)

Step-by-step

219. Install the tools

```
apt update && apt install -y curl netcat-openbsd openssl
```

220. Test Layer 4 reachability (TCP handshake)

```
nc -zv example.com 443
```

221. Test the Layer 6 TLS handshake

```
echo | openssl s_client -connect example.com:443 -servername example.com  
2>/dev/null | head -20
```

222. Test Layer 7 HTTP

```
curl -v https://example.com 2>&1 | head -30
```

223. Inspect timing breakdown

```
curl -w "DNS:%{time_namelookup} TCP:%{time_connect} TLS:%{time_appconnect}  
TTFB:%{time_starttransfer} TOTAL:%{time_total}\n" -o /dev/null -s  
https://example.com
```

224. Apply the layered decision tree

Test it

Verify each layer's tool isolates the failing stage from TCP up to the application.

Note: Full commands and expected output are in labs/lab-36-*.md. Reset the Killercoda playground before the next lab if this one changed firewall, routing or interface state.

Exam Preparation

- First pass: do every lab on Killercoda, reading the explanation under each step.
- Second pass: redo the labs from memory until the command verbs are automatic.
- Memorise the seven OSI layers, the common ports (20/21/22/23/25/53/67-68/80/443...) and the seven troubleshooting steps.
- The N10-009 exam is up to 90 questions in 90 minutes; the passing score is 720 on a 100–900 scale.
- Take the Tertiary Infotech Network+ practice exam:
<https://exams.tertiaryinfotech.com/practice-exams/comptia/comptia-network-plus>
- Book the exam through Pearson VUE from your CompTIA account.

Quick Command Reference

- **ip -brief addr** — Show interfaces, state and IPv4/IPv6 addresses.
- **tcpdump -i eth0 -nn** — Capture packets on an interface without name resolution.
- **dig / nslookup** — Query DNS records (A, AAAA, MX, NS, PTR...) for troubleshooting.
- **ip route / route -n** — Show the routing table and the selected next hop.
- **ping / traceroute / mtr** — Test reachability, path and per-hop latency.
- **arp -n / ip neigh** — Show the IP-to-MAC neighbour (ARP) table.
- **ss -tulpn / netstat -tulpn** — List listening TCP/UDP services and their ports.
- **nmap -sT <host>** — Discover open TCP ports on a target.
- **ufw / iptables** — Configure the host firewall and packet-filter ACLs.
- **systemctl status <svc>** — Check whether a network service is running.

Glossary

- **OSI model** — A seven-layer reference model (Physical → Application) describing network communication.
- **Encapsulation** — Adding a layer's header (and trailer) to data as it moves down the stack.
- **Subnet mask / CIDR** — Bits that split an IP address into network and host portions (e.g. /24).
- **IPv6 SLAAC** — Stateless Address Autoconfiguration — a host derives its own IPv6 address.
- **Port** — A 16-bit number identifying a service (e.g. 80 = HTTP, 443 = HTTPS).
- **DNS** — Domain Name System — resolves names to IP addresses via a hierarchy of zones.
- **DHCP / DORA** — Dynamic Host Configuration Protocol; leases addresses via Discover-Offer-Request-Ack.
- **VLAN / 802.1Q** — A logically separate LAN carried over a shared switch using a VLAN tag.
- **Trunk** — A switch link that carries traffic for multiple tagged VLANs.
- **Link aggregation / LACP** — Bonding several links into one for bandwidth and redundancy.
- **NAT / PAT** — Network/Port Address Translation — maps private addresses to public ones.
- **Administrative distance** — A rank that a router uses to choose between routing sources.
- **SNMP** — Simple Network Management Protocol — polls and traps device health via MIBs.
- **NetFlow** — A protocol that exports summarised traffic-flow records for analysis.
- **Syslog / SIEM** — Centralised logging; a SIEM correlates events for security monitoring.
- **NTP** — Network Time Protocol — synchronises device clocks.
- **CIA triad** — Confidentiality, Integrity, Availability — the goals of security.
- **Firewall / ACL** — Rules that permit or deny traffic by address, port or protocol.
- **IDS / IPS** — Intrusion Detection/Prevention System — inspects traffic for threats.
- **TLS** — Transport Layer Security — encrypts data in transit (HTTPS).
- **VPN** — Virtual Private Network — an encrypted tunnel (site-to-site or client-to-site).
- **MTU / PMTUD** — Maximum Transmission Unit and Path-MTU Discovery for packet sizing.

Support

- Training provider: Tertiary Infotech Academy Pte Ltd (UEN 201200696W).
- Contact: enquiry@tertiaryinfotech.com · +65 6100 0613 · www.tertiarycourses.com.sg

- Course materials and the assessment are on the LMS: <https://lms-tms.tertiaryinfotech.com/>

Assessment flow

On assessment day, follow this order:

- TRAQOM — scan the TRAQOM QR code on the LMS and complete the survey.
- Take the Assessment digital attendance.
- Sit the Written Assessment (WA), then the Practical Performance (PP) — both open book.
- Submit your completed answers on the LMS.
- Sign the Assessment Summary Record.